Mining Massive Data (SS2026)

Programming Assignment 2

Large-scale SVM Training

Availability date: 10.04.2026
Submission due date: 07.05.2026, 23:59
Peer-review due date: 14.05.2026

Number of tasks: 1
Max. achievable points: 100 points (70+20+10 ... Submission / Reviews / Quality of your review)

General Remarks

- This is one of 3 programming assignments. For each assignment you can earn up to 100 points. Details on grading are provided below.
- The deadline for this assignment is noted on the header. No deadline extensions will be granted. Upload your solution by the deadline on the course's Moodle page [2]. If you fail to upload your solution by the deadline you will not be awarded any points.
- If you have questions or encounter problems, do not hesitate to contact us.
- Solve this programming assignment in Python. Don't use any non-standard libraries in your code, e.g., only use `numpy`, `scipy`, `scikit-learn`, unless explicitly mentioned.
- Make sure that your code is executable, e.g., if you created a jupyter notebook, make sure that the notebook can be executed without error when you reset the kernel and execute the notebook from the beginning. **If your code is not executable, we reserve the right to not award any points.**
- Solve the tasks on your own (considered group-wise). If we observe plagiarism across groups, we reserve the right to not award any points.
- Pack your code, your report, and execution instructions into a single .zip file with the following name: `group(group number).zip` file (e.g., `group01.zip`) and upload it on Moodle.
- Only **one** team-member submits the zip file on Moodle.

Questions

Please ask questions primarily on the respective Moodle forum. If you don't get a response in reasonable time (96 hours), get in touch with us via email (Alexander Sturm, Sebastian Tschiatschek).

Support Vector Machines using SGD

Goal

The goal of this assignment is to explore optimization strategies for scaling supervised learning to massive datasets, with a focus on both optimization techniques and kernel approximations. You will study and compare Support Vector Machines (SVMs) trained using different variants of Stochastic Gradient Descent (SGD) and approximate kernel methods. The emphasis is on understanding the trade-offs between accuracy, runtime, and resource usage.

The assignment consists of two parts.

- **Part 1: Scalable Optimization — From Online to Mini-batch Learning**
  You will train linear SVMs using different variants of stochastic gradient-based optimization, and analyze how batching strategies and adaptive methods affect convergence and generalization.

  Specifically, you will

  - compare vanilla SGD, SGD with momentum, and Adagrad, both in
  - *Online mode* (one datapoint per update) and *Mini-batch mode*, with varying batch sizes (e.g., 8, 32, 128)

  You will evaluate how these settings impact:

  - Convergence speed (passes through the data)
  - Training time
  - Accuracy

  Last but not least, you should reflect on how mini-batch size influences the trade-off between computation per step and gradient noise, a crucial aspect in modern

large-scale systems.

- **Part 2: Scaling Kernel Methods with Random Fourier Features (RFFs)**

  To explore nonlinear decision boundaries at scale, you will implement Random Fourier Features (RFFs) to approximate kernelized SVMs efficiently.

  You will

  - use RFFs to approximate the RBF kernel,
  - study the effect of the number of features on accuracy, training time, and memory use, and
  - compare the performance of these models to the linear SVMs from Part 1.

Datasets

In this assignment you will consider different datasets for which we provide an overview in Table 1. The datasets have been selected to cover a range of scales, domains, and formats to give you practical experience with scaling up supervised learning. All datasets are available via Moodle.

Name	**Domain**	**Format**	**Purpose**
`toydata.csv`	Synthetic	Tabular, dense	Debugging, visualization
`toydata_large.csv`	Synthetic	Tabular, dense	Runtime/accuracy comparison
`IMDB` [1]	Text (sentiment)	Sparse (TFIDF)	Real-world, high-dimensional
`higgs.csv`	Physics (tabular)	Dense, large-scale	Runtime scalability, batch size effects

Table 1: Overview of datasets used in the assignment.

Development Dataset: The smallest dataset, `toydata.csv`, is designed to help you develop and debug your implementation. It is ideal for quick visualization and sanity checks.

Medium-scale Dataset: The second dataset, `toydata_large.csv`, is a synthetic but larger dataset meant to reveal differences in runtime and convergence behavior across optimization algorithms and batch sizes. You can also compare the effects of using original features versus kernel approximations (e.g., RFFs).

Real-world Sparse Dataset: The third dataset, `IMDB`, is based on movie reviews labeled as positive or negative sentiment. We provide a preprocessed version in bag-of-words format with TFIDF weighting. This dataset is high-dimensional and sparse. It is stored in a `.npz` file which you can load using:

```
import numpy as np
data = np.load("imdb.npz", allow_pickle=True)
X_train, y_train = data["train"], data["train_labels"]
X_test, y_test = data["test"], data["test_labels"]
```

Note: The feature matrices are in a sparse format. Refer to the `scipy.sparse` documentation for working with these structures.

Higgs Dataset: The Higgs dataset (`higgs.csv`) is a large tabular dataset from high-energy physics. It is useful for exploring optimization and batching strategies at scale.

This dataset can be loaded similarly as the IMDB dataset.

Objective function and performance metrics

For your implementation, you can use the multi-class hinge loss (if you want):

$$\ell(\mathbf{w}^0, \ldots, \mathbf{w}^K; \mathbf{x}, y) = \max\{0, 1 + \max_{j \in \{0,\ldots,K\}, j \neq y} \mathbf{w}^{j,T}\mathbf{x} - \mathbf{w}^{y,T}\mathbf{x}\}, \tag{1}$$

where $\mathbf{w}^0, \ldots, \mathbf{w}^K$ are weights of the SVM for K different classes, $\mathbf{x}$ are the features of a sample and y is the corresponding label. For cases in which a dataset has binary labels, you can use the above equation with $K = 2$.

For all datasets except IMDB and Higgs, use 5-fold cross validation for computing performances, and use classification accuracy as measure of prediction performance. For IMDB and Higgs, there is explicit training and test data, and you can split off a validation set from the training data if needed. For Higgs, report the AUC instead of the classification accuracy.

Assignment

Implement an SVM in the primal form (hinge-loss + ℓ_2 regularization), and train it using various scalable optimization methods (details below). You will explore both linear and kernelized models, and compare the impact of optimizer variants, kernel approximations, and mini-batch configurations. Your experiments will be conducted on the four above-mentioned datasets of increasing complexity and size, including a high-dimensional text dataset (IMDB) and a large-scale tabular dataset from particle physics (Higgs).

Linear SVM with SGD Variants (25%). Implement a linear SVM trained with mini-batch Stochastic Gradient Descent (SGD). Experiment with three optimizer variants: standard SGD, SGD with momentum, and Adagrad.

Train a linear SVM on each dataset (`toydata.csv`, `toydata_large.csv`, `imdb.npz`, `higgs.npz`), selecting a suitable learning rate and learning rate schedule (see the lecture notes for inspiration), regularization parameter, momentum (if applicable), and batch size.

☐ Report how you selected the learning rate, learning rate schedule, batch size, and regularization parameter. Did you use different parameters for different datasets or optimizer variants?

☐ Include convergence plots for the optimizers (training loss vs. number of passes/epochs).

☐ Report classification accuracy/AUC and runtime (e.g., number of epochs, total wall-clock time) for each dataset and optimizer variant.

☐ Compare and discuss the effects of optimizer variants and batch sizes. What trends do you observe?

☐ Briefly describe your implementation of each optimizer.

Random Fourier Features (RFF) for Kernel SVM (25%). Use Random Fourier Features (RFFs) to approximate a Gaussian (RBF) kernel for the SVM. Train the SVM using the same optimization methods (SGD, SGD with momentum, Adagrad).

☐ Briefly describe your implementation of RFFs.

☐ Report how you selected the number of RFFs (minimum 3 different values, at least one ≥ 100), learning rate, and regularization parameter.

☐ Include convergence plots for one optimizer across different RFF sizes.

☐ Report classification accuracy/AUC and runtime for each dataset. Discuss how the number of RFFs affects accuracy and runtime.

☐ For IMDB and Higgs, also report performance on different training set sizes (e.g., 1000, 2000, 3000 samples). Compare your results to using `sklearn.svm.SVC` (as far as this is feasible—remark if things break).

Hint: If the whole dataset (in the case of IMDB or Higgs) is too large for you to process, cut it to a sensible (large) size. Remark on this in your report.

Mini-batch Size and Online vs. Mini-batch Learning (20%). Investigate how different mini-batch sizes impact the performance of your SVMs. For the Higgs dataset, using a linear SVM and an SVM with RFFs, compare results across online learning (batch size = 1), small batches (e.g., 32), and large batches (e.g., 512).

☐ Report training convergence, accuracy/AUC, and runtime for at least three mini-batch sizes for each optimizer (standard SGD, momentum, Adagrad).

☐ Discuss how mini-batch size affects convergence speed, optimization stability, and final accuracy. Do these effects vary across datasets?

Report Write a report summarizing your implementation, experimental setup, and findings. Include the required tables, figures, and discussion as indicated above. Your report should demonstrate an understanding of the trade-offs involved in scalable supervised learning with linear and kernelized models. As always, your report can be part of a well-readble jupyter notebook.

All datasets are available via Moodle. See the Datasets section for details on how to load each file.

References

[1] Andrew L. Maas et al. "Learning Word Vectors for Sentiment Analysis". In: *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*. Ed. by Dekang Lin, Yuji Matsumoto, and Rada Mihalcea. Portland, Oregon, USA: Association for Computational Linguistics, June 2011, pp. 142–150. URL: https://aclanthology.org/P11-1015.

[2] *Mining Massive Data Moodle Page*. URL: https://moodle.univie.ac.at/course/view.php?id=461528.